Лабораторна Робота №5
Тема: Сегментація зображень

Об’єднує: Тема 8

- Використання U-Net або Segment Anything (SAM);
- Виділення об’єктів і заміна фону.

Результат: практичне вміння сегментувати зображення.

1. Використання U-Net або Segment Anything (SAM)

Сегментація зображень є одним з ключових завдань комп’ютерного бачення, де метою є не просто знайти об’єкти, а точно визначити, які саме пікселі належать кожному об’єкту на сцені. На відміну від класифікації (де визначається один клас для всього зображення) або детекції (де об'єкти оточуються обмежувальними рамками), сегментація забезпечує найвищий рівень деталізації — розподіл зображення на структуровані області. У цій лабораторній ми розглянемо сучасні підходи та моделі для сегментації, зокрема архітектуру U-Net та універсальну модель Segment Anything (SAM), які дозволяють отримувати точні маски об’єктів на зображеннях. Також практично буде показано, як виділяти людей, фон та інші об’єкти і виконувати заміну фону, що є прикладом реального прикладного застосування сегментації у дизайні, фотообробці, AR/VR та мультимедіа сервісах.

Для початку роботи встановіть необхідні залежності:

```
pip install torch torchvision albumentations opencv-python matplotlib
```

1.1. Архітектура U-Net

1.1.1. Мінімальна U-Net + аугментації + луп тренування

Першим прикладом, який ми розглянемо ініціалізація моделі з архітектурою U-Net та її навчання. У результаті виконання даного прикладу, ви отримаєте:

- навчену модель U-Net (файл unet_bin.pth) – це ваги моделі, яка навчилась сегментувати певний об’єкт/клас у нашому датасеті;
- значення loss та IoU після кожної епохи – ви побачите як модель навчається і покращує якість з часом;
- розуміння пайплайну сегментації даних, а саме як читати маски, як робити аугментацію та нормалізацію, тобто повний класичний цикл тренування сегментації;
- готову модель для inference, яка може потім створювати маску на новому зображенні.

Для навчання моделі будемо використовувати набір даних у папці *dataset*, в якій містяться папки із зображеннями (*images*), на яких будемо тренувати модель та бінарні маски (*masks*) 0/255 з тими самим іменами файлів.

Створіть новий файл, у якому будемо створювати модель U-Net з назвою unet.py.

Далі у новому файлі почніть введення коду із імпортування бібліотек необхідних для виконання даного прикладу:

```
import torch, torch.nn as nn
```

U-Net будується з багатьох однакових повторюваних блоків, які ми назвемо CBR (Convolution + BatchNorm + ReLU), що дозволяють:

- робити локальне витягування ознак;
- нормалізувати активації;
- тримати здорову стабільність градієнтів.

```
def CBR(in_ch, out_ch):
    return nn.Sequential(
        nn.Conv2d(in_ch, out_ch, 3, padding=1),
        nn.BatchNorm2d(out_ch),
        nn.ReLU(inplace=True),
        nn.Conv2d(out_ch, out_ch, 3, padding=1),
        nn.BatchNorm2d(out_ch),
        nn.ReLU(inplace=True),
 )
```

Після цього ініціалізуємо саму модель U-Unet:

```
class UNet(nn.Module):
    def __init__(self, in_ch=3, out_ch=1):
        super().__init__()
        self.enc1 = CBR(in_ch, 64)
        self.enc2 = CBR(64, 128)
        self.enc3 = CBR(128, 256)
        self.enc4 = CBR(256, 512)
        self.pool = nn.MaxPool2d(2)
        self.bottleneck = CBR(512, 1024)
        self.up4 = nn.ConvTranspose2d(1024, 512, 2, 2)
        self.dec4 = CBR(1024, 512)
        self.up3 = nn.ConvTranspose2d(512, 256, 2, 2)
        self.dec3 = CBR(512, 256)
        self.up2 = nn.ConvTranspose2d(256, 128, 2, 2)
        self.dec2 = CBR(256, 128)
        self.up1 = nn.ConvTranspose2d(128, 64, 2, 2)
        self.dec1 = CBR(128, 64)
        self.outc = nn.Conv2d(64, out_ch, 1)

    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))
        e3 = self.enc3(self.pool(e2))
        e4 = self.enc4(self.pool(e3))
        b  = self.bottleneck(self.pool(e4))
        d4 = self.up4(b); d4 = self.dec4(torch.cat([d4, e4], dim=1))
        d3 = self.up3(d4); d3 = self.dec3(torch.cat([d3, e3], dim=1))
        d2 = self.up2(d3); d2 = self.dec2(torch.cat([d2, e2], dim=1))
        d1 = self.up1(d2); d1 = self.dec1(torch.cat([d1, e1], dim=1))
        return self.outc(d1)
```

Далі створіть новий файл, у якому будемо власне працювати з моделлю. У новому файлі почніть введення коду із імпортування бібліотек необхідних для виконання даного прикладу:

```
import os, cv2, numpy as np, torch, torch.nn as nn
```

```
from torch.utils.data import Dataset, DataLoader
import albumentations as A
from albumentations.pytorch import ToTensorV2
from unet import UNet
```

Зверніть увагу на останній рядок, в якому ми імпортуємо модель U-Net, яку створили попередньо.

Спочатку ініціалізуємо конфігурацію для тренування та попередню обробку зображень:

```
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
IMG_SIZE = 256
BATCH = 8
EPOCHS = 10
DATA_DIR = "dataset"

train_tf = A.Compose([
    A.Resize(IMG_SIZE, IMG_SIZE),
    A.HorizontalFlip(p=0.5),
    A.RandomBrightnessContrast(p=0.2),
    A.Normalize(),
    ToTensorV2(),
])
mask_tf = A.Compose([
    A.Resize(IMG_SIZE, IMG_SIZE)
])
```

Далі створюємо клас SegDataset - це спеціальний клас датасету для PyTorch, який відповідає за завантаження пар зображення + маска для задачі сегментації, іншими словами – це місток між файловою системою та тренуванням моделі: він бере зображення та відповідну маску, правильно їх парує, читає, перетворює, аугментує та віддає у модель у форматі тензорів:

```
class SegDataset(Dataset):
    def __init__(self, img_dir, mask_dir):
        self.img_paths = sorted([os.path.join(img_dir, f) for f in
os.listdir(img_dir)])
        self.mask_dir = mask_dir
    def __len__(self): return len(self.img_paths)
    def __getitem__(self, i):
        ip = self.img_paths[i]; mp = os.path.join(self.mask_dir,
os.path.basename(ip))
        img = cv2.cvtColor(cv2.imread(ip), cv2.COLOR_BGR2RGB)
        mask = cv2.imread(mp, cv2.IMREAD_GRAYSCALE)  # 0/255
        mask = (mask>127).astype(np.float32)  # 0/1
        augi = train_tf(image=img); img_t = augi["image"]
        augm = mask_tf(image=mask); mask_t =
torch.tensor(augm["image"]).unsqueeze(0)
        return img_t, mask_t
```

Наступним кроком готуємо дані до тренування: створюємо об’єкт датасету SegDataset, який буде читати зображення та відповідні маски з папок, формуємо батчі з датасету (ds), які потім будуть подаватися у модель під час тренування, створюємо модель U-Net, готуємо функцію втрат, створюємо оптимізатор AdamW:

```
ds = SegDataset(f"{DATA_DIR}/images", f"{DATA_DIR}/masks")
dl = DataLoader(ds, batch_size=BATCH, shuffle=True, num_workers=0)

model = UNet(in_ch=3, out_ch=1).to(DEVICE)
crit  = nn.BCEWithLogitsLoss()
opt   = torch.optim.AdamW(model.parameters(), lr=1e-3)
```

Також, маємо ініціалізувати функцію для обчислення IoU (Intersection over Union) для сегментації — тобто міру того, наскільки передбачена маска моделі співпадає з реальною правильною маскою:

```
def iou_score(pred, target, eps=1e-6):
    pred = (torch.sigmoid(pred)>0.5).float()
    inter = (pred*target).sum(dim=(1,2,3))
    union = (pred+target - pred*target).sum(dim=(1,2,3))
    return ((inter+eps)/(union+eps)).mean().item()
```

І останнім кроком запускаємо тренування моделі на вказану кількість епох та зберігаємо натреновану модель:

```
for epoch in range(1, EPOCHS+1):
    model.train(); loss_sum=0; iou_sum=0
    for x,y in dl:
        start_time_iteration = time()
        x,y = x.to(DEVICE), y.to(DEVICE)
        opt.zero_grad()
        logits = model(x)
        loss = crit(logits, y)
        loss.backward(); opt.step()
        loss_sum += loss.item()*x.size(0)
        iou_sum  += iou_score(logits.detach(), y)*x.size(0)
    print(f"Epoch {epoch}: loss={loss_sum/len(ds):.4f},
IoU={iou_sum/len(ds):.4f}")
torch.save(model.state_dict(), "unet_bin.pth")
```

В результаті тренування моделі, ми будемо мати в консолі логи, які показують результати функцію втрат та показник IoU. Отриману модель ми будемо використовувати в наступних прикладах. Проаналізуйте отримані результати.

1.1.2. Інференс + візуалізація

Після того як модель U-Net була навчена, наступним кроком є виконання інференсу — застосування моделі до нових зображень, яких вона не бачила під час тренування. На цьому етапі студент вчиться завантажувати навчений файл ваг, подавати зображення у модель з тим самим препроцесом, що використовувався при навчанні, отримувати прогнозовану маску та візуалізувати результат поверх оригінального зображення. Ця частина демонструє, як уже готова сегментаційна модель може працювати в реальній практиці та як оцінити якість виходу не тільки через метрики, а й через безпосередній візуальний аналіз.

Створіть новий файл, в якому будемо працювати з моделлю. У новому файлі почніть введення коду із імпортування бібліотек необхідних для виконання даного прикладу:

```
import cv2, torch, numpy as np
from unet import UNet
```

Далі готуємо конфігурацію для підготовки зображень та ініціалізації моделі:

```
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
IMG_SIZE=256

model = UNet(3,1).to(DEVICE)
model.load_state_dict(torch.load("unet_bin.pth", map_location=DEVICE))
model.eval()
```

Далі додайте код, який виконує завантаження зображень, нормалізацію та подання її в модель U-Net, отримання прогнозованої маски сегментації, бінаризацію її по порогу та повернення до початкового розміру зображення:

```
img = cv2.cvtColor(cv2.imread("1_test_images/1.jpg"),
cv2.COLOR_BGR2RGB)
orig = img.copy()
img_r = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
t =
torch.from_numpy((img_r/255.0).transpose(2,0,1)).float().unsqueeze(0).to(DEVICE)
with torch.no_grad():
    m = torch.sigmoid(model(t))[0,0].cpu().numpy()
    logits = model(t)
    s = torch.sigmoid(logits)
m = (m>0.3).astype(np.uint8)*255
m = cv2.resize(m, (orig.shape[1], orig.shape[0]),
interpolation=cv2.INTER_NEAREST)
```

Останнім кроком, додайте код, який накладає отриману маску на вихідне зображення для візуалізує результат сегментації та додатково зберігає heatmap (інтенсивність передбачення моделі) у окремий файл:

```
overlay = cv2.addWeighted(
    cv2.cvtColor(orig, cv2.COLOR_RGB2BGR),
    1.0,
    cv2.merge([np.zeros_like(m), m, np.zeros_like(m)]),
    0.5,
    0
)
cv2.imwrite("unet_mask_overlay.jpg", overlay)

heat = (s[0,0].cpu().numpy()*255).astype(np.uint8)
cv2.imwrite("heat.png", heat)
```

В результаті ми отримаємо два зображення, одне з яких – це вхідне зображення з накладеною маскою, яке розпізнала модель. Перше зображення, це одне з тих, на яких навчали модель, щоб перевірити чи модель була добре навчена. Далі спробуйте прогнати модель на інших зображеннях з папки *1_test_images*. Проаналізуйте результати. Прогляньте зображення, на яких модель була навчена та знайдіть схожі зображення. Спробуйте, чи модель буде розпізнавати маску. Проаналізуйте результати.

1.1.3. Підрахунок площі маски інференсу

У цьому прикладі ми показуємо, що сегментація — це не лише виділення об’єкта на зображенні, а й можливість переходу до кількісного аналізу. Після отримання маски за допомогою U-Net ми можемо обчислити площу об’єкта в пікселях та, при наявності масштабного коефіцієнта, перевести її в реальні фізичні одиниці. Такий підхід широко використовується у практичних застосуваннях — у медицині, агровізуальному аналізі,

промисловому контролі, де важливо не тільки знайти об’єкт, а й виміряти його характеристику.

В цілому код для даного прикладу дуже схожим із попереднім прикладом, адже містить той самий шматок коду для інференсу зображення. Створіть новий файл для початку роботи на прикладом та додайте код з імпортом бібліотек необхідних для виконання даного прикладу:

```
import cv2, torch, numpy as np
from unet import UNet
```

Далі готуємо конфігурацію для підготовки зображень та ініціалізації моделі:

```
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
IMG_SIZE=256

model = UNet(3,1).to(DEVICE)
model.load_state_dict(torch.load("unet_bin.pth", map_location=DEVICE))
model.eval()
```

Далі додайте код, який виконує завантаження зображень, нормалізацію та подання її в модель U-Net, отримання прогнозованої маски сегментації, бінаризацію її по порогу та повернення до початкового розміру зображення:

```
img = cv2.cvtColor(cv2.imread("1_test_images/1.jpg"),
cv2.COLOR_BGR2RGB)
orig = img.copy()
img_r = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
t =
torch.from_numpy((img_r/255.0).transpose(2,0,1)).float().unsqueeze(0).to(DEVICE)
with torch.no_grad():
    m = torch.sigmoid(model(t))[0,0].cpu().numpy()

m = (m>0.3).astype(np.uint8)*255
m = cv2.resize(m, (orig.shape[1], orig.shape[0]),
interpolation=cv2.INTER_NEAREST)
```

Останнім кроком, додайте код, який виконує розрахунок площі сегментованої ділянки та створює зображення із позначеною сегментованою ділянкою:

```
area_pixels = np.sum(m > 127)
print("Площа об'єкта в пікселях:", area_pixels)

PIX_AREA_MM2 = 0.25
real_area = area_pixels * PIX_AREA_MM2
print("Реальна площа приблизно (мм^2):", real_area)

cv2.imwrite("mask_area.png", m)
```

В результаті ми отримаємо зображення із позначеною сегментованою ділянкою та виведенням у консолі отриманої розрахованої площі цієї ділянки. Далі спробуйте прогнати модель на інших зображеннях з папки *1_test_images*. Проаналізуйте результати. Прогляньте зображення, на яких модель була навчена та знайдіть схожі зображення. Спробуйте, чи модель буде розпізнавати маску. Проаналізуйте результати. Чи на вашу думку модель правильно сегментувала ділянку та отриману площу можна вважати дійсною?

1.2. Архітектура SAM

Segment Anything Model (SAM) — це універсальна модель сегментації від Meta, яка дозволяє виділяти об'єкти на зображенні без попереднього донавчання на конкретному датасеті. Для отримання сегментації достатньо лише підказки від користувача — точки, рамки чи кількох точок, і SAM автоматично будує маску потрібного об'єкту. Це робить SAM зручним та швидким інструментом для розмітки даних, створення масок, підготовки датасетів, а також для інтерактивних експериментів, коли потрібно отримати сегментацію "на льоту" без довгого тренування моделей.

Для початку роботи встановіть необхідні залежності:

```
pip install opencv-python matplotlib torch torchvision
pip install git+https://github.com/facebookresearch/segment-
anything.git
```

Використання prompt-точок у SAM потрібне для того, щоб інтерактивно керувати тим, який саме об'єкт ми хочемо сегментувати на зображенні. У реальних сценах часто є багато різних елементів, і модель не може автоматично знати, що саме користувач хоче виділити. Тому клацання точкою(ами) дає моделі мінімальну, але дуже точну підказку — "ось тут знаходиться потрібний об'єкт". Це дозволяє отримувати сегментацію максимально швидко, точно та без складної ручної розмітки, що робить SAM дуже ефективним інструментом для розмітки даних, підготовки масок та досліджень сегментації без довгої підготовки датасетів. Такий підхід називається "prompt-based segmentation" і дозволяє дуже швидко та зручно отримувати якісні маски без ручного виділення контуру або попереднього тренування.

1.2.1. Використання prompt-точок

Далі ми розглянемо такий приклад, в якому використовуються вже заздалегідь визначені точки.

Створіть новий файл для початку роботи на прикладом та додайте код з імпортом бібліотек необхідних для виконання даного прикладу:

```
import cv2, torch, numpy as np
from segment_anything import sam_model_registry, SamPredictor
```

Далі готуємо конфігурацію для підготовки зображень та ініціалізації моделі:

```
sam_ckpt = "sam_vit_h_4b8939.pth"
model_type = "vit_h"
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
POINT_OF_INTEREST = [400, 320]
```

де *sam_ckpt* – шлях до checkpoint файлу (.pth) з вагами моделі, *model_type* – який саме варіант архітектури SAM використовується, *POINT_OF_INTEREST* – наша наперед визначена точка.

Далі додайте код, який ініціалізує модель, обгортку навколо SAM моделі яка робить інференс простішим і надає інтерфейс для prompt-based сегментації та зображення:

```
sam = sam_model_registry[model_type](checkpoint=sam_ckpt)
sam.to(device=DEVICE)
predictor = SamPredictor(sam)

img = cv2.cvtColor(cv2.imread("1_test_images/1.jpg"),
cv2.COLOR_BGR2RGB)
```

```
predictor.set_image(img)

input_points = np.array([POINT_OF_INTEREST])
input_labels = np.array([1])
```

Далі додайте код, який запускає сам процес передбачення, розпізнавання та накладання маски на об’єкт і зберігає зображення з маскою:

```
masks, scores, _ = predictor.predict(
    point_coords=input_points,
    point_labels=input_labels,
    multimask_output=True
)
best = masks[np.argmax(scores)]
mask = (best*255).astype(np.uint8)

overlay = cv2.addWeighted(
    cv2.cvtColor(img, cv2.COLOR_RGB2BGR),
    1.0,
    cv2.merge([np.zeros_like(mask), mask, np.zeros_like(mask)]),
    0.5,
    0
)
cv2.imwrite("sam_point_overlay.jpg", overlay)
```

Виконайте збережений код. Проаналізуйте результати. Чи була накладена маска правильно? Спробуйте використати інші зображення з тестової папки *1_test_images* або будь-які свої та інші точку з координатами та проаналізуйте отримані результати.

1.2.2. Використання prompt-точок

Далі розберемо приклад, більш інтерактивний, коли користувач може самостійно вибрати об’єкт на зображенні, клацнувши по ньому.

Створіть новий файл для початку роботи на прикладом та додайте код з імпортом бібліотек необхідних для виконання даного прикладу, також код, який ініціалізує модель, обгортку навколо SAM моделі яка робить інференс простішим і надає інтерфейс для prompt-based сегментації та зображення з попереднього прикладу, але видаліть наступні рядки:

```
POINT_OF_INTEREST = [400, 320]
input_points = np.array([POINT_OF_INTEREST])
input_labels = np.array([1])
```

Замість видалених рядків будемо використовувати код, який відкриє спочатку вікно із зображенням, щоб ви могли клацнути на об’єкт, який вас цікавить, що модель одразу перейшла до процесу передбачення. Для цього спочатку додайте код функції для зворотного виклику події на клацання по зображенні:

```
points = []
def click_event(event, x, y, flags, param):
    if event == cv2.EVENT_LBUTTONDOWN:
        points.append([x,y])
        cv2.destroyAllWindows()
```

Наступним кроком, додайте код для показу зображення, яке ми будемо використовувати для передбачення:

```
cv2.imshow("img", orig)
cv2.setMouseCallback("img", click_event)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Далі, вкажіть код, який відповідає за початок процесу передбачення, використовуючи координати, які ми отримали після клацання по зображенню та формування маски:

```
print(f"Вибрано точок: {points}")
input_point = np.array(points)
input_label = np.ones(len(points), dtype=np.int32)

masks, scores, _ = predictor.predict(
    point_coords=input_point,
    point_labels=input_label,
    multimask_output=True
)
best = masks[np.argmax(scores)]
mask = (best*255).astype(np.uint8)
```

І останнім кроком буде додавання коду, для виведення та збереження вхідного зображення із накладеною маскою:

```
overlay = cv2.addWeighted(
    cv2.cvtColor(img, cv2.COLOR_RGB2BGR),
    1.0,
    cv2.merge([np.zeros_like(mask), mask, np.zeros_like(mask)]),
    0.5,
    0
)
cv2.imwrite("sam_point_overlay.jpg", overlay)
cv2.imshow("Image with overlay", overlay)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Збережіть даний файл та виконайте його. У результаті виконання, спочатку має з’явитися вікно із зображенням, в якому ми маєте клацнути мишкою по об’єкті, який ви хочете використати для накладання маски. Далі з’явиться друге вікно із готовим результатом. Проаналізуйте результати. Чи була накладена маска правильно? Спробуйте використати інші зображення з тестової папки *1_test_images* або будь-які свої для накладання маски та проаналізуйте отримані результати.

1.2.3. Автоматичний генератор масок

Окрім самостійного вибору об’єкта для створення маски на зображенні, ми можемо задати SAM моделі автоматичну сегментацію та накладання масок. У цього прикладі ми розглянемо саме такий випадок, а саме модель сегментує та накладе всі маски, проте ми виведемо та покажемо тільки найбільшу за площею.

Створіть новий файл для початку роботи на прикладом та додайте код з імпортом бібліотек необхідних для виконання даного прикладу, проте додайте ще один новий імпорт бібліотеки модуля os та імпорт інших функцій бібліотеки SAM:

```
import os
```

```
from segment_anything import sam_model_registry,
SamAutomaticMaskGenerator
```

Далі додайте код для ініціалізації назви та типу моделі SAM, яку будемо використовувати:

```
sam_ckpt = "sam_vit_h_4b8939.pth"
model_type = "vit_h"
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
```

Наступний кроком, додайте код для ініціалізації самого екземпляру моделі SAM:

```
sam = sam_model_registry[model_type](checkpoint=sam_ckpt).to(DEVICE)
generator = SamAutomaticMaskGenerator(sam, points_per_side=16,
pred_iou_thresh=0.86)
```

Зверніть увагу, що в даному випадку ми використовуємо метод *SamAutomaticMaskGenerator*, який відповідає за автоматичну сегментацію зображення.

Далі додайте код, який завантажує зображення, обробляє його(переводить у градації сірого) та власне запускає автоматичних генератор сегментації та отримує список масок:

```
orig = cv2.imread("1_test_images/2.jpg")
img = cv2.cvtColor(orig, cv2.COLOR_BGR2RGB)
masks = generator.generate(img)
print("N masks:", len(masks))
```

Додайте також код, який створить папку sam_auto, якщо такої не існує:

```
if not os.path.exists("sam_auto"):
    os.makedirs("sam_auto")
```

та збереже всі сегментовані маски у створеній папці:

```
for i, m in enumerate(masks):
    cv2.imwrite(f"sam_auto/sam_auto_mask_{i}.png",
(m['segmentation'].astype(np.uint8))*255)
```

І останнім кроком, додайте код, який вибере зображення із найбільшою маскою за площею, виведе це зображення та збереже його окремо:

```
largest = max(masks, key=lambda m: m['area'])
largest_image = (largest['segmentation'].astype(np.uint8))*255
cv2.imwrite("sam_auto_largest.png", largest_image)
cv2.imshow("Image with overlay", largest_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Збережіть файл та виконайте його. Що було отримано у результаті, проаналізуйте їх. Спробуйте також виконати код, використовуючи інші тестові зображення та проаналізуйте результати.

1.2.4. Автоматичний генератор масок (усі маски кольорові на одному зображенні)

В окремих випадках нам може бути потрібно зображення з усіма накладеними масками.

Для цього використаємо попередній код. Скопіюйте файл з попереднього прикладу та вкажіть нове ім’я для файлу. У новому файлі, в кінці додайте наступний код:

```
vis = orig.copy()
colors = [(int(255*np.random.rand()), int(255*np.random.rand()),
int(255*np.random.rand())) for _ in range(len(masks))]
```

```
for i, m in enumerate(masks):
    mask = (m['segmentation'].astype(np.uint8))*255
    color = colors[i]
    overlay = vis.copy()
    overlay[mask>0] = color
    vis = cv2.addWeighted(vis, 0.6, overlay, 0.4, 0)

cv2.imwrite("sam_color_masks_overlay.png", vis)
cv2.imshow("Image with overlay", vis)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Збережіть файл та виконайте його. У результаті виконання виконаються ті ж самі дії та збережуться ті самі файли, що і в попередньому випадку, проте останнім покажеться зображення з усіма масками у різних кольорах. Окрім того, дане зображення збережеться окремо.

2. Виділення об’єктів і заміна фону

Виділення об’єктів і заміна фону є однією з ключових практичних задач комп’ютерного зору, тісно пов’язаною із сегментацією зображень. Її мета полягає у точному виокремленні об’єкта на зображенні від фону, після чого фон може бути модифікований, видалений або замінений іншим зображенням. Така задача широко застосовується у фотомонтажі, AR/VR, відеообробці, рекламній індустрії, стрімінгу, робототехніці та інструментах автоматичного редагування фото. Моделі глибокого навчання на кшталт U-Net або сучасних універсальних сегментаторів, таких як SAM (Segment Anything Model), дозволяють виконувати це завдання з високою точністю, автоматизуючи процес та зменшуючи залежність від ручного виділення областей.

Першим етапом ми створимо файл для утиліт, які будемо використовувати у наступних прикладах. Створіть новий файл із назвою *utils_mask.py* та наступним кодом:

```
import cv2, numpy as np
import os, shutil

def refine_mask(mask, k_open=3, k_close=5, blur=5):
    """mask: uint8 (0/255). Морфологія + легке розмивання краю."""
    m = mask.copy()
    if k_open:
        m = cv2.morphologyEx(m, cv2.MORPH_OPEN,
cv2.getStructuringElement(cv2.MORPH_ELLIPSE,(k_open,k_open)))
    if k_close:
        m = cv2.morphologyEx(m, cv2.MORPH_CLOSE,
cv2.getStructuringElement(cv2.MORPH_ELLIPSE,(k_close,k_close)))
    if blur:
        m = cv2.GaussianBlur(m, (blur|1, blur|1), 0)
    return m

def alpha_composite(fg_bgr, bg_bgr, alpha_0_255):
    """Композит: out = a*fg + (1-a)*bg; alpha у [0..255]."""
    a = (alpha_0_255.astype(np.float32) / 255.0)[..., None]
```

```
            fg = fg_bgr.astype(np.float32)
            bg = cv2.resize(bg_bgr, (fg.shape[1],
fg.shape[0])).astype(np.float32)
            out = a*fg + (1-a)*bg
            return out.clip(0,255).astype(np.uint8)

        def keep_subject_blur_bg(img_bgr, mask_255, k=21):
            """Залишити суб’єкт чітким, фон розмити."""
            blur = cv2.GaussianBlur(img_bgr, (k|1, k|1), 0)
            return alpha_composite(img_bgr, blur, mask_255)

        def prepare_folder(folder_name):
            if os.path.exists(folder_name) and os.path.isdir(folder_name):
                shutil.rmtree(folder_name)
            if not os.path.exists(folder_name):
                os.makedirs(folder_name)
```

Дані методи виконують наступні дії:

- *refine_mask* — прибере «дірки» й зробить м’який край;
- *alpha_composite* — накладання вирізаного об’єкта на фон;
- *keep_subject_blur_bg* — популярний ефект «портретного режиму»;
- *prepare_folder* — видалить попередню папку, якщо така існує за вказаним ім’ям та створить нову.

2.1.Архітектура U-Net

2.1.1. Заміна фону із U-Net

Для початку роботи з даним прикладом створіть новий файл та додайте наступний код для імпорту необхідних залежностей:

```
        import cv2, torch, os, numpy as np
        from unet import UNet
        from utils_mask import refine_mask, alpha_composite,
keep_subject_blur_bg, prepare_folder
```

Далі додайте код, яким ми ініціалізуємо основні змінні для прикладу:

```
        DEVICE   = "cuda" if torch.cuda.is_available() else "cpu"
        IMG_SIZE = 256
        WEIGHTS  = "unet_bin.pth"
        IMG      = "1_test_images/1.jpg"
        BG       = None
        RESULT_FODLER = "8_unet_results"
        THRESHOLD = 0.2
```

Зверніть увагу, у цьому прикладі ми використовуємо модель з вагами *unet_bin.pth*, яку ми навчали у попередній частині лабораторної. Також будемо використовувати для першого прикладу зображення, яке ми використовували для навчання, також вказуємо фон *None* на який будемо замінювати маску та папку, в якій будемо зберігати результати *RESULT_FODLER* та поріг *THRESHOLD* сегментації маски.

Далі, додайте код, який ініціалізує модель та завантажує ваги із навченої моделі:

```
        model = UNet(3,1).to(DEVICE)
        state = torch.load(WEIGHTS, map_location=DEVICE)
        model.load_state_dict(state, strict=False)
```

```
model.eval()
```

Наступним кроком, додайте код для завантаження зображення та її попередньої обробки та процесом сегментації:

```
img_bgr = cv2.imread(IMG);  assert img_bgr is not None
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
src_h, src_w = img_rgb.shape[:2]
img_res = cv2.resize(img_rgb, (IMG_SIZE, IMG_SIZE))
x =
torch.from_numpy((img_res.astype(np.float32)/255.0).transpose(2,0,1)).unsqueeze(0)
.to(DEVICE)

with torch.no_grad():
    s = torch.sigmoid(model(x))[0,0].cpu().numpy()
```

Після цього виділяємо та формуємо маску:

```
mask_small = (s>THRESHOLD).astype(np.uint8)*255
mask = cv2.resize(mask_small, (src_w, src_h),
interpolation=cv2.INTER_NEAREST)
mask = refine_mask(mask, k_open=3, k_close=5, blur=7)
```

Відповідно до налаштувань замінюємо маску на зображення або просто білий колір:

```
if BG is None:
    bg_bgr = np.full_like(img_bgr, 255)
else:
    bg_bgr = cv2.imread(BG);  assert bg_bgr is not None

replaced = alpha_composite(img_bgr, bg_bgr, mask)
portrait  = keep_subject_blur_bg(img_bgr, mask, k=31)
```

І додайте останній блок коду для збереження отриманих результатів:

```
prepare_folder(RESULT_FOLDER)
cv2.imwrite(os.path.join(RESULT_FOLDER, "unet_mask.png"), mask)
cv2.imwrite(os.path.join(RESULT_FOLDER, "unet_replace_bg.png"),
replaced)
cv2.imwrite(os.path.join(RESULT_FOLDER, "unet_blur_bg.png"), portrait)
print("Saved: unet_mask.png, unet_replace_bg.png, unet_blur_bg.png")
```

Збережіть файл та виконайте його. Оскільки ми тренували модель на зображенні (Рис. 2.1), ми протестуємо це зображення першим.

Рисунок 2.1. Одне з зображень, яке використовувалося для тренування моделі

У результаті виконання коду, в нашій папці мають створитися три зображення: розмите зображення, яке являє собою популярний ефект «портретного режиму», зображення із сегментованою маскою, та зображення із де ми замінили фон з маскою (Рис. 2.2).

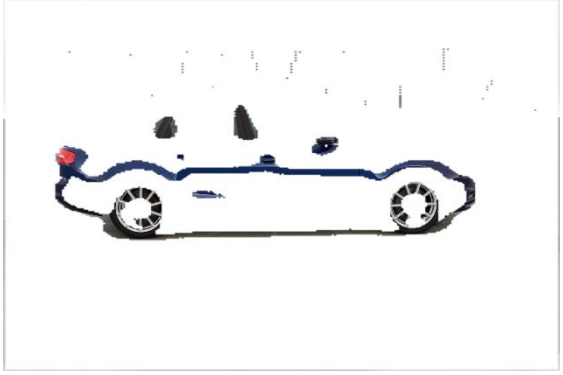

Рисунок 2.2. Те саме зображення, але з вирізаним фоном за маскою

Як можна помітити, отриманий результат, не зовсім коректний і може бути не тим, на що могли бути очікування. Можна спробувати змінити поріг впевненості *THRESHOLD* на менше значення, напр. 0.18 (Рис. 2.3).

Рисунок 2.3. Те саме зображення, але з вирізаним фоном за маскою

Цього разу результат дещо інший, проте всеодно може бути далеким від очікуваного. Це пояснюється тим, що тренувальний набір зображень та масок, міг бути не зовсім коректним, було обрано замало/забагато кількість епох для тренування, показник тренування *LEARNING_RATE = 3e-4* міг теж бути замалим/завеликим.

Спробуйте поекспериментувати із значенням *THRESHOLD*, щоб досягти найкращого результату. Після визначення оптимального значення THRESHOLD протестуйте його також на інших зображеннях із папки *1_test_images*, а також проекспериментуйте з різними значеннями цього показника для інших прикладів, щоб оцінити вплив параметра на результат. Проаналізуйте результати. Чи отриманий найкращий показник *THRESHOLD* підходить і для інших зображень? Які показники *THRESHOLD* забезпечили найкращі результати для інших зображень?

2.2. Архітектура SAM

Як ми вже розглянули в попередній частині лабораторної, модель SAM дозволяє задавати координати, або ж якщо така можливість була додана, вибирати самостійно об'єкт на зображенні для його сегментації. Така функціональність може бути використана в тому числі і для обробки зображень (фотографій) для видалення/заміни небажаних об'єктів на зображенні чи заміни фону.

2.2.1. Заміна фону використовуючи модель SAM

Як і у випадку з U-Net, першим прикладом, який ми розглянемо буде заміна маски фону на білий колір або інше зображення.

Для початку роботи з даним прикладом створіть новий файл та додайте наступний код для імпорту необхідних залежностей:

```
import cv2, torch, os, numpy as np
from unet import UNet
from utils_mask import refine_mask, alpha_composite,
keep_subject_blur_bg, prepare_folder
```

Далі додайте код, яким ми ініціалізуємо основні змінні для прикладу:

```
IMG = "1_test_images/2.jpg"
BG  = None
SAM_WEIGHTS = "sam_vit_h_4b8939.pth"
MODEL_TYPE  = "vit_h"
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
RESULT_FOLDER = "9_sam_results"
```

Зверніть увагу, у цьому прикладі ми використовуємо зображення *2.jpg* (Рис. 2.4) в якому модель має самостійно визначити маску (в перспективі саме фон) та замінити його на *None*, що в даному випадку буде означати заміну фону на білий колір. Інші параметри налаштування є стандартними для такої задачі. Зберігати зображення будемо у папці *9_sam_results*.

Далі додайте код, який буде завантажувати зображення для попередньої обробки:

```
img_bgr = cv2.imread(IMG)
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```

Наступним етапом буде додання коду відповідального за ініціалізацію моделі та задання підготовленого зображення для опрацювання:

```
sam = sam_model_registry[MODEL_TYPE](checkpoint=SAM_WEIGHTS).to(DEVICE)
predictor = SamPredictor(sam)
predictor.set_image(img_rgb)
```

Далі, вказуємо код, який запускає передбачення моделі та отримує готову маску:

```
h, w = img_bgr.shape[:2]
box = np.array([int(0.1*w), int(0.1*h), int(0.9*w), int(0.9*h)])
```

```
masks, scores, _ = predictor.predict(box=box, multimask_output=True)
mask = masks[np.argmax(scores)].astype(np.uint8)*255
mask = refine_mask(mask, k_open=3, k_close=5, blur=7)
```

Рисунок 2.4. Тестове зображення

Додайте код в якому ми готуємо зображення або колір фону, який будемо вставляти у зображенні:

```
if BG is None:
    bg_bgr = np.full_like(img_bgr, 255)
else:
    bg_bgr = cv2.imread(BG);  assert bg_bgr is not None

replaced = alpha_composite(img_bgr, bg_bgr, mask)
portrait  = keep_subject_blur_bg(img_bgr, mask, k=31)
```

І останнім етапом, додайте код для збереження результатів для відображення отриманого зображення:

```
prepare_folder(RESULT_FOLDER)
cv2.imwrite(os.path.join(RESULT_FOLDER, "sam_mask.png"), mask)
cv2.imwrite(os.path.join(RESULT_FOLDER, "sam_replace_bg.png"),
replaced)
cv2.imwrite(os.path.join(RESULT_FOLDER, "sam_blur_bg.png"), portrait)
cv2.imshow("Image with replaced background", replaced)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

print("Saved: sam_mask.png, sam_replace_bg.png, sam_blur_bg.png")

Збережіть файл та виконайте його. В результаті виконання даного коду, ми отримаємо зображення (Рис. 2.5), в якому фон буде замінено на білий колір. Спробуйте виконати даний код замінивши зображення у змінній IMG на інше зображення з тестової папки. Проаналізуйте отримані результати. Чи був правильно визначено фон та замінено його на білий колір?

Рисунок 2.5. Тестове зображення із заміненим фоном на білий колір

2.2.2. Заміна фону використовуючи модель SAM на інший фон

В даному прикладі ми будемо замінювати фон не на білий колір а на інше зображення. Для цього ми будемо використовувати код із попереднього прикладу.

Замініть значення змінної BG на backgrounds/1.jpg, збережіть файл та виконайте код. У результаті буде отримано зображення із заміненим фоном із підготовленого зображення (Рис. 2.6).

Рисунок 2.6. Тестове зображення із заміненим фоном на інше зображення

Протестуйте даний приклад також на інших зображеннях із папки *1_test_images* та використовуючи інші зображення із папки *backgrounds* чи з власними зображеннями. Проаналізуйте отримані результати.

2.2.3. Видалення об’єктів із зображення (інпейнтинг) використовуючи модель SAM

Далі розглянемо інший приклад використання моделі SAM видалення об’єкту з використанням маски, при цьому не замінюючи маску об’єкту на інше зображення, а використовуючи інпайтинг. Інпайтинг — це процес заповнення або заміни відсутніх або

пошкоджених частин зображення для створення повного та безшовного результату. Іншими словами, ми спробуємо замінити маску об’єкта, щоб повністю видалити об’єкт і заповнити місце навколишнім фоном. При цьому ми надамо можливість користувачу самостійно вибирати об’єкт для видалення та інпайтинга.

Для початку роботи з даним прикладом створіть новий файл та додайте наступний код для імпорту необхідних залежностей:

```
import os, cv2, numpy as np, torch
from segment_anything import sam_model_registry, SamPredictor
from utils_mask import prepare_folder
```

Далі додайте код, яким ми ініціалізуємо основні змінні для прикладу:

```
IMG_PATH    = "10_test_images/1.jpg"
SAM_WEIGHTS = "sam_vit_h_4b8939.pth"
MODEL_TYPE  = "vit_h"
DEVICE      = "cuda" if torch.cuda.is_available() else "cpu"
RESULT_FOLDER = "10_sam_results"
```

Далі додайте код для завантаження та попередньої обробки тестового зображення 1.jpg (Рис. 2.7):

```
img_bgr = cv2.imread(IMG_PATH)
assert img_bgr is not None
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```

Рисунок 2.7. Тестове зображення

Далі додайте код для ініціалізації моделі SAM:

```
sam = sam_model_registry[MODEL_TYPE](checkpoint=SAM_WEIGHTS).to(DEVICE)
predictor = SamPredictor(sam)
predictor.set_image(img_rgb)
```

Наступним кроком, додайте код для зворотного виклику функції для збереження координат для виділення об’єкта на зображенні, ініціалізації та виклику тестового зображення:

```
points = []
def click_event(event, x, y, flags, param):
    if event == cv2.EVENT_LBUTTONDOWN:
        points.append([x,y])
        cv2.destroyAllWindows()
```

```
cv2.namedWindow('Select object on Image', cv2.WINDOW_NORMAL)
cv2.resizeWindow('Select object on Image', 800, 600)
cv2.imshow("Select object on Image", img_bgr)
cv2.setMouseCallback("Select object on Image", click_event)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Далі нам потрібно підготувати отримані координати для використання:

```
print(f"Вибрано точок: {points}")
input_point = np.array(points)
input_label = np.ones(len(points), dtype=np.int32)
```

Підготувавши всі необхідні дані, запускаємо модель для передбачення та виділення маски об’єкта по заданим координатам:

```
masks, scores, _ = predictor.predict(
    point_coords=input_point,
    point_labels=input_label,
    multimask_output=True
)
best = masks[np.argmax(scores)]
mask = (best*255).astype(np.uint8)
```

У результаті ми отримаємо маску об’єкта, який хочемо видалити використавши інпайтинг. Для цього, спочатку потрібно задати параметри для інпайтингу, де ми створюємо ядро для інпайтингу розміром 9x9 пікселів у формі еліпса та використовуємо операцію дилатація (dilate) для розширення білих областей на масці:

```
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (9,9))
mask_dilated = cv2.dilate(mask, kernel, iterations=1)
```

При інпайнтингу дуже часто по краю об’єкта залишається тонкий "ореол" (boundary halo) після маскування, особливо коли SAM обрізає дуже точно по контуру. Дилатація дає моделі інпайнтингу трохи більший контур, відповідно більше контексту пікселів, що означає більш плавне і якісне вирізаного об’єкта.

Далі запускаємо саму процедуру інпайтингу:

```
result_telea = cv2.inpaint(img_bgr, mask_dilated, inpaintRadius=10,
flags=cv2.INPAINT_TELEA)
result_ns    = cv2.inpaint(img_bgr, mask_dilated, inpaintRadius=10,
flags=cv2.INPAINT_NS)
```

Зверніть увагу на параметр inpaintRadius. InpaintRadius — це радіус області (в пікселях) навколо маски, з якої алгоритм «позичає» інформацію для відновлення (заповнення дірки): малий inpaintRadius (3–5) бере інформацію дуже близько до краю, при цьому виконання алгоритму швидке, але часто сильніше розмазує, може давати артефакти на складних текстурах; великий inpaintRadius (10–25) забезпечує більше контексту, краще відновлення складних текстур (цегла, сходи, дерево...), але повільніше, і може «втягувати» більше фону.

І останнім кроком, додайте код, який зберігає результат та виводить отримане зображення після інпайтингу:

```
prepare_folder(RESULT_FOLDER)
cv2.imwrite(os.path.join(RESULT_FOLDER, "sam_obj_mask.png"), mask)
cv2.imwrite(os.path.join(RESULT_FOLDER, "sam_mask_dilated.png"),
mask_dilated)
```

```
        cv2.imwrite(os.path.join(RESULT_FOLDER, "removed_telea.png"),
result_telea)
        cv2.imwrite(os.path.join(RESULT_FOLDER, "removed_ns.png"), result_ns)
        print("Saved: sam_obj_mask.png, sam_mask_dilated.png,
removed_telea.png, removed_ns.png")
        cv2.namedWindow('Image with inpainting', cv2.WINDOW_NORMAL)
        cv2.resizeWindow('Image with inpainting', 800, 600)
        cv2.imshow("Image with inpainting", result_ns)
        cv2.waitKey(0)
        cv2.destroyAllWindows()
```

Збережіть файл та виконайте його. Спочатку з’явиться вікно, в якому клацніть по об’єкту, який хочете видалити із зображення, наприклад, клацніть по коту, який знаходиться найближче. Після цього вікно закриється та алгоритм почне процедуру інпайтингу. В результаті, ви отримаєте результат схожий до Рис. 2.8.

Рисунок 2.8. Тестове зображення після інпайтингу

Як можна помітити, модель правильно визначили об’єкт для видалення. Проте результат не надто відповідає очікуванням, а саме повна відповідність маски фону, тобто замість «розмитої плями» очікувалося відображення правильного фону. Спробуйте використати інші значення для радіусу інпайтингу і інші зображення із тестової папки *10_test_images* та проаналізуйте результати.

В цілому, такий результат цілком очікуваний, так як в даному випадку самої моделі SAM та OpenCV недостатньо для такої задачі. Для таких сцен із виразною текстурою краще працюють глибинні інпайнтинги: LaMa / ZITS / Stable Diffusion Inpainting. Вони відтворюють повторювані патерни (сходинки/цегла) значно реалістичніше, ніж Telea/NS.